

Conceptos básicos — lo que hay que entender antes de tocar la consola

Cinco ideas sostienen todo lo que se construye en los instructivos A y B. Si entendés estas, el resto es configuración. A cada concepto lo acompaña la **analogía de la biblioteca**, que usamos en todo el curso.

1. LLM (modelo de lenguaje) — y por qué no alcanza solo

Un LLM (Gemini, en nuestro caso) es un modelo entrenado para predecir y generar texto. Sabe muchísimo de lo que vio en su entrenamiento, pero **no conoce tus datos específicos y actuales** (la agenda de tu evento, tu archivo histórico). Y si se le pregunta algo que no sabe, puede **inventar una respuesta convincente**. Eso se llama **alucinación** y es el riesgo número uno de un asistente Q&A.

En la biblioteca: un redactor brillante recién llegado a la ciudad — escribe perfecto, pero no sabe nada del barrio. Y jamás dice "no sé".

2. RAG: buscar primero, responder después

RAG (Retrieval-Augmented Generation) es la técnica que evita las alucinaciones. En vez de dejar que el modelo conteste de memoria, primero se **buscan los fragmentos relevantes en tus documentos** (Retrieval) y luego se le pide que **redacte la respuesta usando esos fragmentos** (Augmented Generation). El modelo deja de adivinar y pasa a responder con material concreto.

En la biblioteca: el bibliotecario no opina de memoria — va al estante, trae el libro y te lee lo que dice.

3. Grounding y citas: anclar la respuesta a la fuente

Grounding es el resultado de aplicar RAG: la respuesta queda **anclada a un documento concreto** y, normalmente, **lo cita**. En la plataforma de Google esto se logra conectando un datastore a tu app o agente. La regla de oro que le damos al asistente: **si no está en los documentos, no lo inventes** — y el "no cuento con esa información" es una función, no una falla.

En la biblioteca: cada respuesta viene con la ficha del libro del que salió. Y si el libro no está en los estantes, el bibliotecario te lo dice.

4. Datastore: la biblioteca

Es el repositorio donde la plataforma **guarda e indexa** tus documentos. Internamente hace por vos todo el trabajo pesado del RAG: parsear cada archivo, partirlo en fragmentos, generar los vectores de significado, indexar y recuperar. Por eso el proyecto no requiere programar: la plataforma gestionada resuelve lo que de otro modo harías a mano (ver "Por dentro", abajo).

En la biblioteca: los estantes + el fichero. **Indexar = fichar los libros** para poder encontrarlos rápido. El bucket de Cloud Storage es el depósito del que la biblioteca toma los documentos para ficharlos.

5. Contexto y ventana de contexto

El contexto es todo lo que el modelo "tiene a la vista" al responder: las instrucciones, los fragmentos recuperados y el **historial de la conversación** (por eso se puede repreguntar "¿y el segundo día?"). La **ventana de contexto** es el límite de cuánto cabe ahí. Importa por dos motivos: si metemos demasiado, sube el costo (se paga por token) y el modelo se distrae; si metemos poco o mal, la respuesta sale pobre. Ojo: el contexto **dura la conversación** — cada visitante nuevo del asistente arranca de cero, y está bien que así sea.

En la biblioteca: el bibliotecario se acuerda de qué venían hablando — pero con cada visitante nuevo, la charla empieza limpia.

Por dentro: el pipeline que la plataforma hace solo

Estos términos aparecen si alguna vez espiás "la caja negra" (o en la demo Python de la guía técnica). La plataforma los gestiona sola, pero conviene conocerlos:

Término	Qué es
Chunking	Partir cada documento en fragmentos manejables (p. ej. 500–1.000 tokens) para poder buscarlos por separado
Embedding	Representar cada fragmento como un vector numérico que captura su significado — textos parecidos quedan cerca en el espacio vectorial
Vector store	Base de datos especializada en guardar esos vectores y encontrar los más parecidos a una pregunta
Retrieval / top-k	Dada una pregunta, recuperar los k fragmentos más cercanos (p. ej. los 4 más relevantes)
Re-ranking	Reordenar esos fragmentos por relevancia fina antes de pasárselos al modelo

La pregunta también se convierte en vector: buscar es "medir distancias de significado", no coincidencia de palabras. Por eso "¿hay algo del sector alimentario?" encuentra un documento que nunca dice "alimentario".

El diccionario del curso (resumen)

Cuando escuches...	Pensá en...
Datastore	La biblioteca (estantes + fichero)
Indexar	Fichar los libros
La app / el agente	El bibliotecario de la ventanilla
Prompt / instrucciones	Las reglas de trabajo del bibliotecario

Cuando escuches...	Pensá en...
Bucket (Cloud Storage)	El depósito del que la biblioteca toma los documentos
Widget	La ventanilla de consultas puesta en la página web
OCR	Transcribir un documento viejo escaneado para poder ficharlo
Alucinación	El redactor que inventa cuando no sabe
Fallback	"No cuento con esa información" — honestidad por diseño

Siguiente paso: con estos conceptos, seguí con el [Instructivo A](#) (la app con widget, sin código) o el [Instructivo B](#) (el agente propio con Agent Studio).